UniTik

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Kevin Makwana
26TBSCIT034

Under the esteemed guidance of
Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

KISHINCHAND CHELLARAM COLLEGE

(Affiliated to HSNC University)
MUMBAI, 400020
MAHARASHTRA
2026 – 27

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN No. : 24031151018 Roll No. : 26TBSCIT034
PRN No. : 24031151050 Roll No. : 26TBSCIT054

Name of the Student: Kevin Makwana

Kavya Shah

Title of the Project: UniTik

Name of the Guide: Dr. Pragati Thawani

Teaching Experience of the Guide: 20 years

Is this your first Submission? Yes ☑ No ☐

Signature of the Student Signature of the Guide

Date: …….. Date: ……..

Signature of the Co – ordinator

Date: ……..

KISHINCHAND CHELLARAM COLLEGE

(Affiliated to HSNC University, Mumbai)

MUMBAI, MAHARASHTRA – 400020

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project entitled, **"GroupCart"**, is bonafide work of Areeba Ansari and Nirvika Katkam bearing 26TBSCIT002, 26TBSCIT026 submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from HSNC University.

Internal Guide **Co – ordinator**

External Examiner

Date: **College Seal**

ACKNOWLEDGEMENT

We would like to take this opportunity to express our sincere gratitude to **Dr. Tejashree V. Shanbhag (Principal of K.C. College)**, and **Dr. Rakhi Gupta (H.O.D. of the B.Sc. I.T. Department)**, for their valuable support, guidance and encouragement throughout the development of our project. We would also like to express our profound gratitude to our project guide, **Dr. Pragati Thawani**, for her valuable guidance, continuous support, constructive suggestions and encouragement throughout the various stages of the project. Her guidance and valuable feedback helped us in understanding the requirements, designing the system and successfully completing the initial phases of our project **"UniTik"**. We are grateful to the faculty members of the B.Sc. I.T. Department for providing us with the necessary knowledge, resources and assistance required for carrying out this project. Their valuable inputs and cooperation have contributed significantly to our learning and development. Lastly, we would like to thank everyone who directly or indirectly supported and encouraged us during the course of this project. Their assistance and cooperation have been invaluable in helping us complete this work successfully.

Kevin Makwana

Kavya Shah

DECLARATION

I hereby declare that the project entitled **"GroupCart"**, undertaken at **K.C. College**, has not been duplicated or submitted to any other university or institution for the award of any degree or qualification. To the best of my knowledge, no other person has submitted this project to any other university or institution.

This project is undertaken in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** and is being submitted as a project work as part of my academic curriculum.

Name and Signature of the Students

TABLE OF CONTENTS

Sr. No.	Title	Page No.
1	Introduction	9
	1.1 Background	10
	1.2 Objectives	10
	1.3 Purpose, Scope and Applicability	11
	1.4 Achievements	11
	1.5 Organization of Report	12
2	Survey of Technology	13
	2.1 Front – End Features	14
	2.2 Back – End Features	15
	2.3 Justification – Why this Technology?	17
3	Requirements and Analysis	19
	3.1 Requirement Specification	20
	3.2 Software Requirements	21
	3.3 Hardware Requirements	22
	3.4 Planning & Scheduling	22
4	System Design	24
	4.1 Data Flow Diagram	25
	4.2 Flow Diagram	27
	4.3 ER Diagram	29
	4.4 Class Diagram	31
	4.5 Use Case Diagram	33
	4.6 Sequence Diagram	36
	4.7 Activity Diagram	38
5	References	40

Sr. No.	List of Tables	Page No.
1	Table 3.1 Functional Requirements	20
2	Table 3.2 Non Functional Requirements	21

3	Table 3.3 Software Requirements	21
4	Table 3.4 Hardware Requirements	22
5	Table 3.5 Planning & Scheduling	23
6	Table 4.1 Data Flow Diagram Symbols	25
7	Table 4.2 Flow Diagram Symbols	27
8	Table 4.3 ER Diagram Symbols	29
9	Table 4.4 Class Diagram Symbols	31
10	Table 4.5 Use Case Diagram Symbols	33
11	Table 4.6 Sequence Diagram Symbols	36
12	Table 4.7 Activity Diagram Symbols	38

Sr. No.	List of Figures	Page No.
1	Fig. 3.1 Gantt Chart	23
2	Fig. 4.1 Data Flow Diagram Level 0	26
3	Fig. 4.2 Data Flow Diagram Level 1	27
4	Fig. 4.3 Flow Diagram	28
5	Fig. 4.4 ER Diagram	30
6	Fig. 4.5 Class Diagram	32
7	Fig. 4.6 Use Case Diagram	34
8	Fig. 4.7 Sequence Diagram	37
9	Fig. 4.8 Activity Diagram	39

Sr. No.	List of Abbreviations	Full Form
1	SDLC	Software Development Lifecycle
2	DOM	Document Object Model
3	API	Application Programming Interface
4	REST	Representational State Transfer
5	NPM	Node Package Manager
6	MVT	Model – View – Template
7	ORM	Object – Relational Mapping
8	AMD	Advanced Micro Devices
9	RAM	Random Access Memory
10	SSD	Solid State Drive

ABSTRACT

UniTik is an advanced web-based campus event and ticketing platform designed to overcome the limitations of traditional, individual-focused registration models. Often, students hesitate to purchase event passes early, waiting for price reductions or promotional deals, which challenges organizers in driving predictable attendance. UniTik addresses this dynamic by introducing collaborative group-booking features, allowing peers to purchase passes together and unlock special group discounts once participation thresholds are met. This community-driven approach fosters higher student engagement and provides organizers with scalable sales opportunities and improved capacity forecasting.

Engineered with robust security and role-based architectures, the platform features distinct, polished portals for students and administrators, complemented by automated digital ticketing and QR-code verification. UniTik serves as a practical, user-friendly solution that modernizes campus event management, making participation more affordable and accessible. By bridging the gap between organizers and attendees, the project highlights the power of web-based digital commerce tools in transforming real-world institutional engagement.

Keywords: Campus Events, Group Booking, Ticket Management, Web-Based Platform, Collaborative Purchasing, Student Engagement, Digital Commerce, Event Ticketing

CHAPTER 1

CHAPTER 1: INTRODUCTION

1.1. Background

Contemporary higher education institutions face significant logistical challenges when coordinating campus-wide events, cultural fests, and corporate recruitment drives. Traditional event administration relies heavily on fragmented systems, manual ticketing, and paper-based attendance tracking. This methodology introduces severe vulnerabilities, including proxy attendance during large-scale events, financial discrepancies due to unverified ticket transfers, and operational chaos stemming from decentralized vendor communication. There is a critical need for an automated, unified ecosystem that bridges the gap between administrators, students, and third-party planners to ensure secure, efficient, and transparent operations.

1.2. Objectives

2. To digitize and secure the event ticketing process, eliminating fraudulent entries through dynamic QR verification.
3. To eradicate proxy attendance by implementing high-precision, location-based geofencing for real-time tracking of volunteers and participants.
4. To automate financial reconciliations and marketplace commission splits for multitenant organizers.
5. To provide a centralized, real-time communication and management dashboard for college administrators and corporate entities.
6. To deliver a seamless, high-engagement mobile and web experience for the student body.

1.3. Purpose

The purpose of UniTik is to replace archaic, manual event management workflows with a sophisticated, polyglot microservices architecture. It aims to serve as a self-governing smart platform that handles hierarchical access control, secure transaction processing, and concurrent location tracking within a single, unified interface.

Scope The scope of UniTik encompasses the end-to-end lifecycle of event management. This includes secure user onboarding via Role-Based Access Control (RBAC), digital ticket generation and distribution, real-time GPS-based attendance verification, and automated fund

distribution between the core platform and individual event organizers. The architecture is designed to scale, with future scope including the integration of Machine Learning pipelines for predictive resource analytics and automated document verification.

Applicability UniTik is strictly applicable to higher education ecosystems, including college committees, student councils, and administrative faculty. Furthermore, its multi-tenant architecture makes it fully applicable to external corporate entities hosting campus placement drives, as well as third-party vendors and event planners requiring a dedicated logistical hub for multi-day fests.

1.4. Achievement

1. Design and deployment of multiple services using a polyglot microservices ecosystem.
2. Generation of cryptographically verifiable tickets for secure QR ticketing.
3. Elimination of proxy attendance using GPS-based geofencing in real time.
4. Automated financial reconciliation with automatic commission splits.
5. Implementation of Active Defense DevSecOps against SQLI, DDoS, and token hijacking.
6. Development of a Progressive Web App (PWA) optimized via Backend-For-Frontend architecture.
7. Enforcement of strict RBAC and hierarchical access for multi-tenant operations.
8. Management of distributed state and infrastructure in cloud-native environments.

1.5. Organization of Report

Chapter 1 - Introduction:

This chapter provides an overview of the proposed project, UniTik. It explains the need for the project by discussing the existing problem and the proposed solution. It also describes the objectives, purpose, scope, applicability, and expected achievements of the system, giving a clear understanding of the project and its significance.

Chapter 2 - Survey of Technologies:

The next chapter presents the survey of technologies that will be used for developing the proposed system. It discusses the technologies selected for the frontend, backend, database,

and other development tools. The chapter also explains the justification for choosing these technologies based on the project requirements, ease of development, performance, and future scalability.

Chapter 3 - Requirements & Analysis:

This chapter defines the functional and nonfunctional requirements of the system. It outlines the specific software and hardware resources necessary for the efficient development, testing, and operation of the UniTik platform.

CHAPTER 2

CHAPTER 2: SURVEY OF TECHNOLOGY

2.1. Front-End Features

React.js

React.js is an open-source JavaScript library developed for building modern and interactive user interfaces. It follows a component-based architecture, allowing developers to create reusable UI components that improve code organization and maintenance.

Some of the key features of React.js include reusable components, virtual DOM, efficient rendering, state management, and simplified user interface development.

TypeScript

TypeScript is a strongly typed programming language that builds on JavaScript by adding static type definitions. It allows developers to catch potential runtime errors during compilation, ensuring highly reliable and maintainable code, which is absolute critical when handling financial and geographic data. Some of the key features of TypeScript include static typing, object-oriented programming features, early error detection, and enhanced developer tooling.

CapacitorJS

CapacitorJS is a cross-platform native runtime that allows developers to build modern web apps that run natively on iOS and Android. It enables a single web codebase to be compiled directly into native applications, drastically reducing development time and maintenance overhead. Some of the key features of CapacitorJS include cross-platform compatibility, native API access, seamless progressive web app (PWA) support, and simplified mobile wrapper capabilities.

2.2. Back-End Features

PHP

PHP is a widely used server-side scripting language known for its exceptional stability in handling web forms and relational database transactions. It features highly mature, heavily documented SDKs for third-party integrations, making it ideal for core business logic. Some of the key features of PHP include extensive database support, cross-platform compatibility, robust security frameworks, and seamless integration with payment gateways.

Golang

Golang (Go) is a statically typed, compiled programming language designed for high concurrency and performance. It utilizes native "Goroutines" to handle thousands of

simultaneous processes, making it exponentially faster and more efficient than traditional stateless scripting languages for heavy workloads. Some of the key features of Golang include high concurrency, fast execution, lightweight threading, memory safety, and excellent support for WebSocket connections.

Python

Python is a high-level, interpreted programming language widely used for web development, data analysis, automation, and artificial intelligence. Its simple syntax and extensive library support make it an excellent choice for developing advanced backend applications. Within this ecosystem, Python is utilized to power the Machine Learning pipelines required for predictive resource analytics, automated document verification, predictive surge pricing, and conversational AI. Some of the key features of Python include simple syntax, object-oriented programming, extensive libraries, and strong community support for AI and ML integrations.

Razorpay Route

Razorpay Route is a payment gateway API designed to automate the routing of high-volume payments to multiple service providers. It facilitates automated split payments, deducting commissions and distributing funds seamlessly. Some of the key features include automated financial reconciliation, multi-tenant payouts, seamless vendor management, and reduced operational overhead.

2.3. Justification - Why this Technology? The selection of appropriate technologies plays an important role in the successful development of the proposed system. The technologies used for UniTik have been chosen based on strict engineering criteria regarding performance, scalability, and economic feasibility.

ReactJS & TypeScript

React's component-based architecture allows for rapid UI development and seamless state management. TypeScript is selected because it enforces strict static typing, catching potential runtime errors during compilation, which is absolute critical when handling financial and geographic data.

CapacitorJS

CapacitorJS is chosen as it enables the single React codebase to be compiled directly into native Android and iOS applications. This drastically reduces development time and maintenance overhead compared to managing separate Swift and Kotlin repositories.

PHP

PHP is selected for the core business logic engine due to its exceptional stability in handling web forms, relational database transactions, and its highly mature, heavily documented SDKs for payment gateway integration.

Golang

Golang is specifically selected to power the high-concurrency geofenced attendance system. Its native "Goroutines" handle thousands of simultaneous location pings and WebSocket connections exponentially faster and more efficiently than traditional stateless scripting languages.

Python

Python is selected as the backend programming language for Artificial Intelligence and Machine Learning integrations due to its simplicity, readability, and extensive library support. It enables the efficient development and integration of ML pipelines for predictive resource analytics, automated document verification, surge pricing models, and conversational AI for user queries.

Razorpay Route

Razorpay Route is selected because it automates the routing of high-volume payments to multiple service providers without any hassles. This eliminates the need for manual financial reconciliation and drastically reduces the operational and legal overhead of managing marketplace vendor payouts.

CHAPTER 3

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1. Requirements Specification

The requirements specification defines the functionalities and quality attributes of the proposed UniTik system.

3.1.1. Functional Requirements

- **User Registration and Onboarding:** Secure user onboarding via Role-Based Access Control (RBAC).
- **Ticket Generation:** Generation and distribution of cryptographically verifiable digital tickets to prevent unauthorized entry.
- **Attendance Tracking:** Real-time GPS-based attendance verification utilizing geofencing to eliminate proxy attendance.
- **Financial Management:** Automated financial reconciliations and commission splits between the platform and multi-tenant organizers via payment webhooks.
- **Notification Management:** Real-time broadcast alerts powered by Firebase Cloud Messaging (FCM) to notify targeted user cohorts of ticket drops or schedule changes.
- **AI & ML Integration:** Utilization of predictive surge pricing, resource analytics, and conversational AI to assist user queries and automate document verification.

3.1.2. Non-Functional Requirements

- **Usability and Platform:** The front-end must operate as a highly responsive, mobilefirst Progressive Web App (PWA) with offline capabilities to securely cache QR code tickets.
- **Scalability and Performance:** The architecture must scale to handle thousands of simultaneous location pings and WebSocket connections using lightweight threading.
- **Security:** The system must enforce strict zero-trust security measures, including realtime threat protection against SQLI, DDoS, and token hijacking (Active Defense DevSecOps).

3.2. Software & Hardware Requirements

The development of the proposed UniTik system requires appropriate software and hardware resources to ensure smooth development, testing, and execution.

3.2.1. Software Requirements

Software	Purpose
Operating Syste (Windows 10/1	Platform for application development and execution of the application.
Visual Studio C	Integrated source code editor used for developing and debugging the application.
ReactJS & TypeScript	Developing the frontend user interface with component-based architecture and strict static typing.
CapacitorJS	Compiling the React codebase directly into native Android and iOS applications.
PHP	Implementing core business logic, managing web forms, relational database transactions, and mature payment gateway integration.
Golang	Powering the geofenced attendance system using native Goroutines for concurrent processing.
Python	Implementing Machine Learning (ML) pipelines for predictive resource analytics, surge pricing, automated document verification, and conversational AI.

Relational Data (SQL)	Storing and securely managing application data, transactional logs, and user records.
Git	Version control and project management.
Google Chrome Microsoft Edge	Testing and running the web application.

3.2.2. Hardware Requirements

Hardware	**Specification**
Processor	Intel Core i3 / AMD Ryzen 3 or higher.
RAM	Minimum 8 GB.
Storage	Minimum 256 GB SSD (or equivalent free storage).
Monitor	Standard display with 1366x768 resolution or higher.
Mobile Device	Equipped with GPS sensors for students and volunteers to utilize the PWA and process location tracking.
Internet Connection	Required for development, package installation, and cloud interactions.

3.4 Planning & Scheduling

Planning involves breaking down the development of UniTik into smaller tasks with specific requirements, dependencies, and deadlines. Key tasks include designing the user interface, developing the Next.js frontend and PHP/PDO backend, designing the Supabase PostgreSQL database, implementing student ticket booking and QR-code generation, building the administrator gate-scanning validation module, integrating the different components, and testing the system. Scheduling ensures that these activities are completed systematically within the planned time frame. A Gantt chart is used to visualize the project schedule, track progress, and identify delays or dependencies that may affect the completion of the project.

Table 3.5 Planning & Scheduling

Task	Start Date	End Date	Duration	Sign
Discussion & Approval of Topic	15.06.2026	19.06.2026	4	
UniTik Campus Dataset & Schema Design	20.06.2026	24.06.2026	4	
Feasibility Study & Abstract	25.06.2026	29.06.2026	4	
Chapter 1 : Introduction	30.06.2026	08.07.2026	8	
Chapter 2 : Survey of Technology	09.07.2026	13.07.2026	4	
Chapter 3 : Requirement Analysis	14.07.2026	22.07.2026	8	
Chapter 4 : System Design	23.07.2026	10.08.2026	18	
Final Documentation & Implementation	19.06.2026	24.08.2026	66	

Sr. No.	Task Name	June 2026			July 2026			August 2026			
		15/06	22/06	29/06	08/07	15/07	22/07	03/08	10/08	17/08	24/08
1	Discussion & Approval of Topic										
2	GroupCart Dataset										
3	Feasibility Study & Abstract										
4	Chapter 1 : Introduction										
5	Chapter 2 : Survey of Technology										
6	Chapter 3 : Requirement Analysis										
7	Chapter 4 : System Design										
8	Final Documentation										

CHAPTER 4

CHAPTER 4: SYSTEM DESIGN

System design is the process of defining the structure, components, relationships, and flow of the UniTik software system based on its security, authentication, and ticketing requirements. It acts as a blueprint for the development of the campus event management platform, ensuring a well-structured, efficient, reliable, and secure environment for both students and administrators.

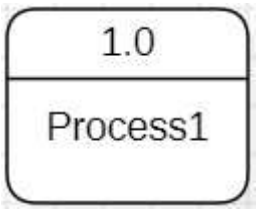
Various diagrams represent different aspects of UniTik, including data flows, database entity relationships, system processes, user activities, and gate verification workflows. These diagrams provide clear, standardized representations to maintain architectural consistency throughout development.

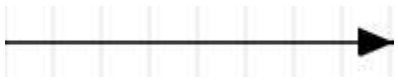
4.1 Data Flow Diagram

A Data Flow Diagram (DFD) visualizes how data moves within the UniTik system between students, administrators, external email services, and the Supabase PostgreSQL database. It maps processes such as authentication requests, ticket bookings, QR code payload generation, and gate scan log entries.

.

Table 4.1 Data Flow Diagram Symbols

Symbol	Name	Description
	External Entity	Represents external users or systems that interact with the GroupCart system, such as Customer and Administrator.
	Process	Represents an operation or function that processes input data to produce output data.
	Data Store	Represents the location where system data is stored for future use or retrieval.

	Data Flow	Represents the movement of data between external entities, processes, and data stores.

4.1.1 DFD Level 0

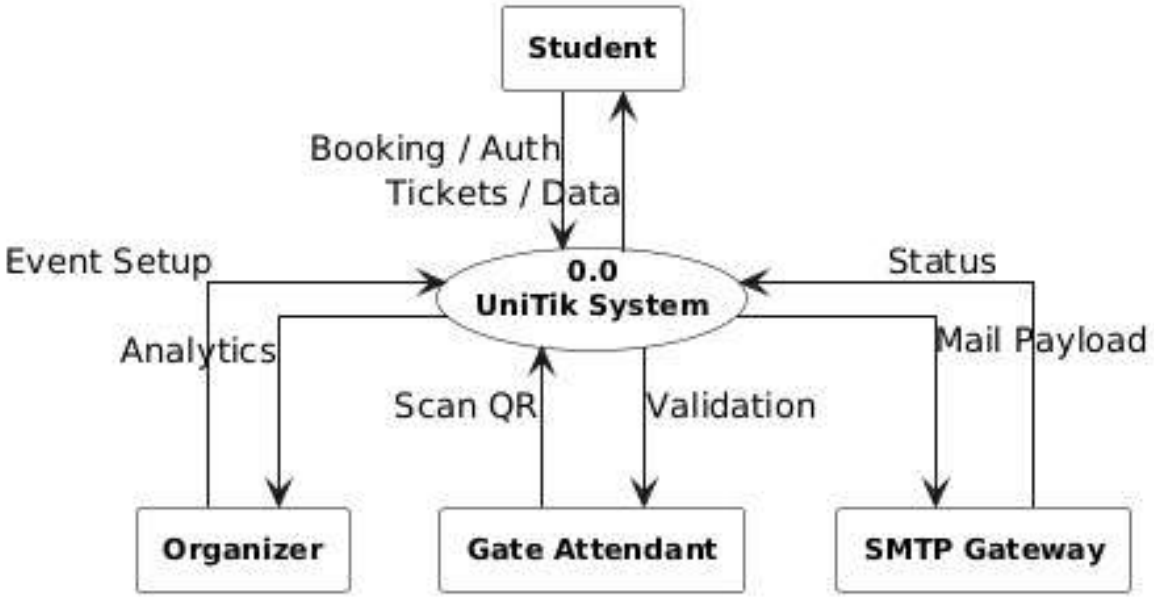


Fig. 4.1 Data Flow Diagram Level 0

4.1.2 DFD Level 1

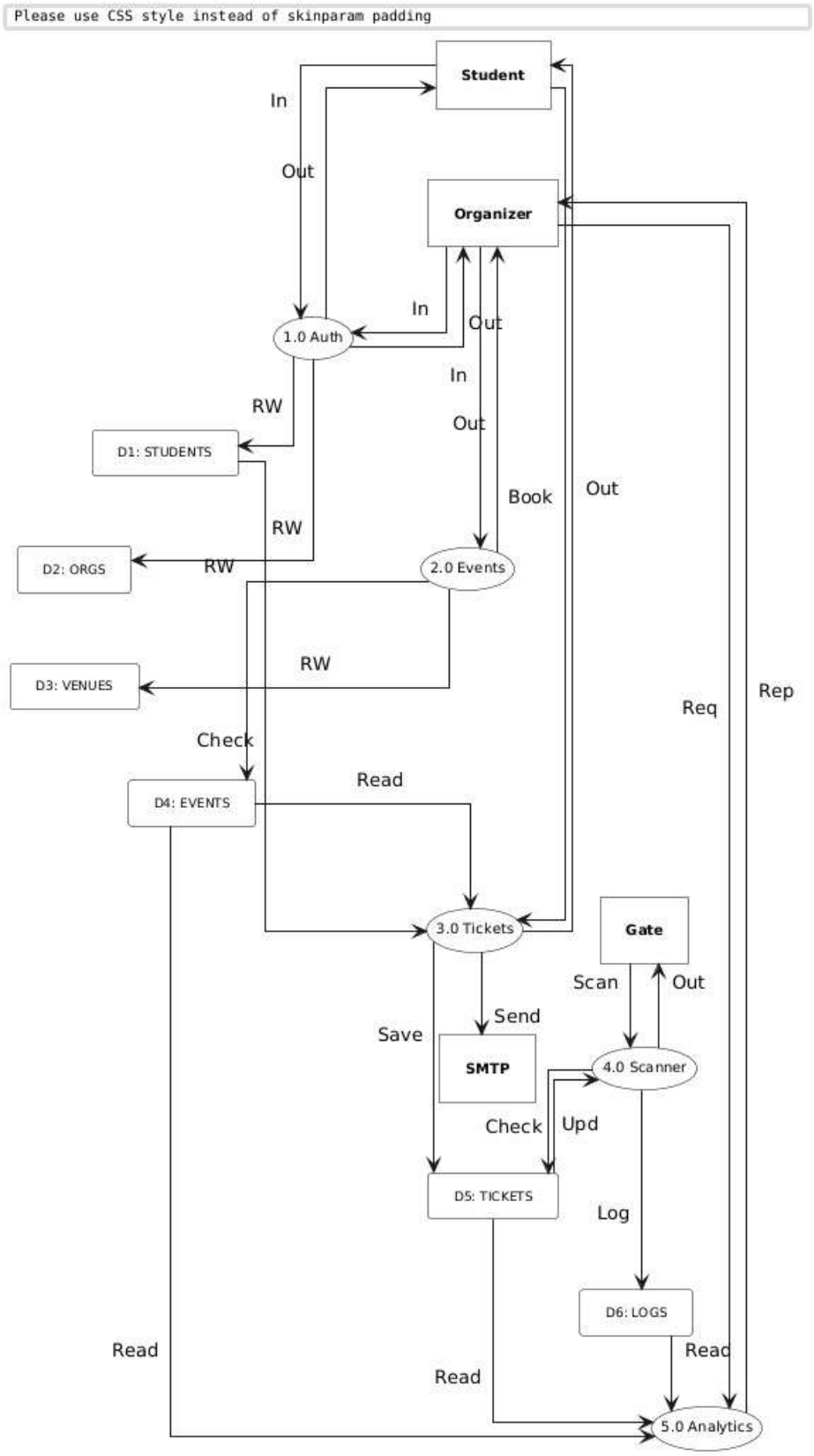
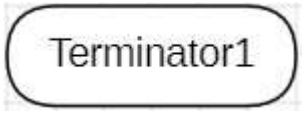
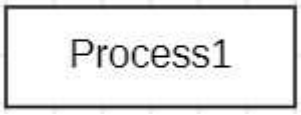
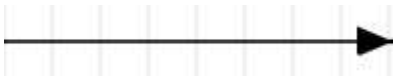


Fig 4.2 Data Flow Diagram Level 1

4.2 Flow Diagram

A Flow Diagram outlines the logical sequence of operations within UniTik, illustrating user navigation from authentication portals through event selection, secure ticket purchasing, and real-time validation at campus entry gates.

Table 4.2 Flow Diagram Symbols

Symbol	Name	Description
	Terminator	Represents the beginning or termination of a process or workflow.
	Process	Represents an operation or activity performed during the workflow.
	Decision	Represents a condition where the flow branches based on different outcomes.
	Flow	Shows the direction and sequence of the workflow.

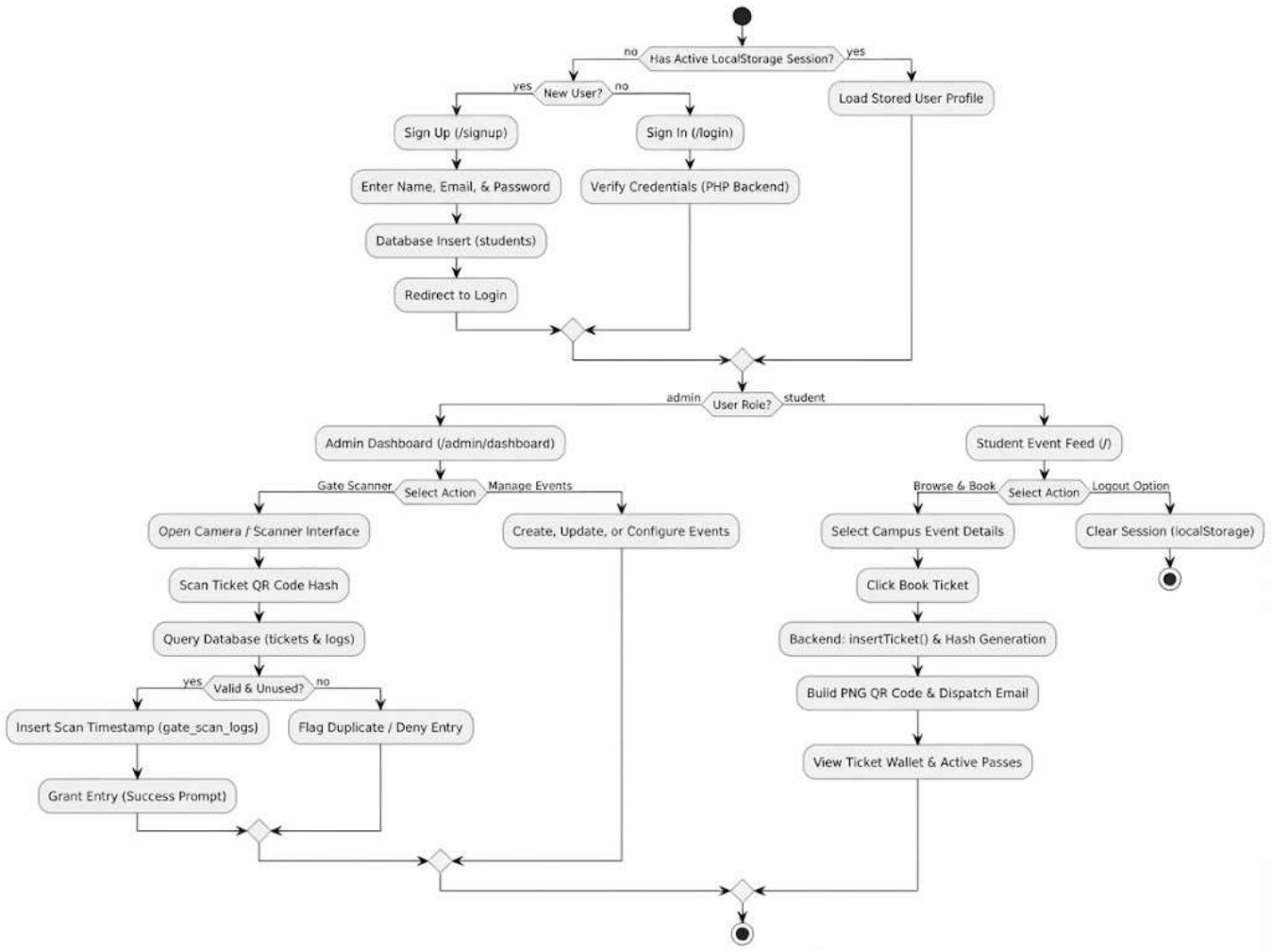


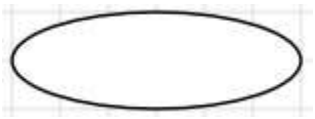
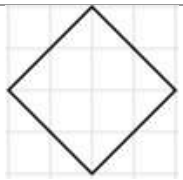
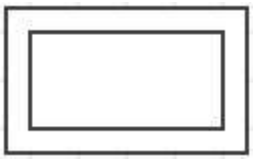
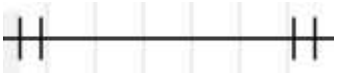
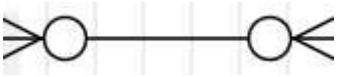
Fig. 4.3 Flow Diagram

4.3 ER Diagram

An Entity Relationship (ER) Diagram defines the logical structure of the UniTik database, highlighting core entities, attributes, and foreign key relationships..

Table 4.3 ER Diagram Symbols

Symbol	Name	Description
Entity1	Entity	An entity represents an object or concept about which information is stored. In an ER diagram, entities are represented using rectangles. Examples include User, Product, Cart, Order, and Payment.

	Attribute	An attribute represents a property or characteristic of an entity. Attributes describe the information associated with an entity, such as UserID, Name, Email, or Price.
	Relationship	A relationship represents how two or more entities are connected. It describes the association between entities within the system.
	Weak Entity	A weak entity is an entity that cannot be uniquely identified by its own attributes alone and depends on another entity for its identification.
	One-to-One	In a one-to-one relationship, one instance of an entity is associated with only one instance of another entity.
	One-toMany	In a one-to-many relationship, one instance of an entity can be associated with multiple instances of another entity.
	Many-toMany	In a many-to-many relationship, multiple instances of one entity can be associated with multiple instances of another entity.

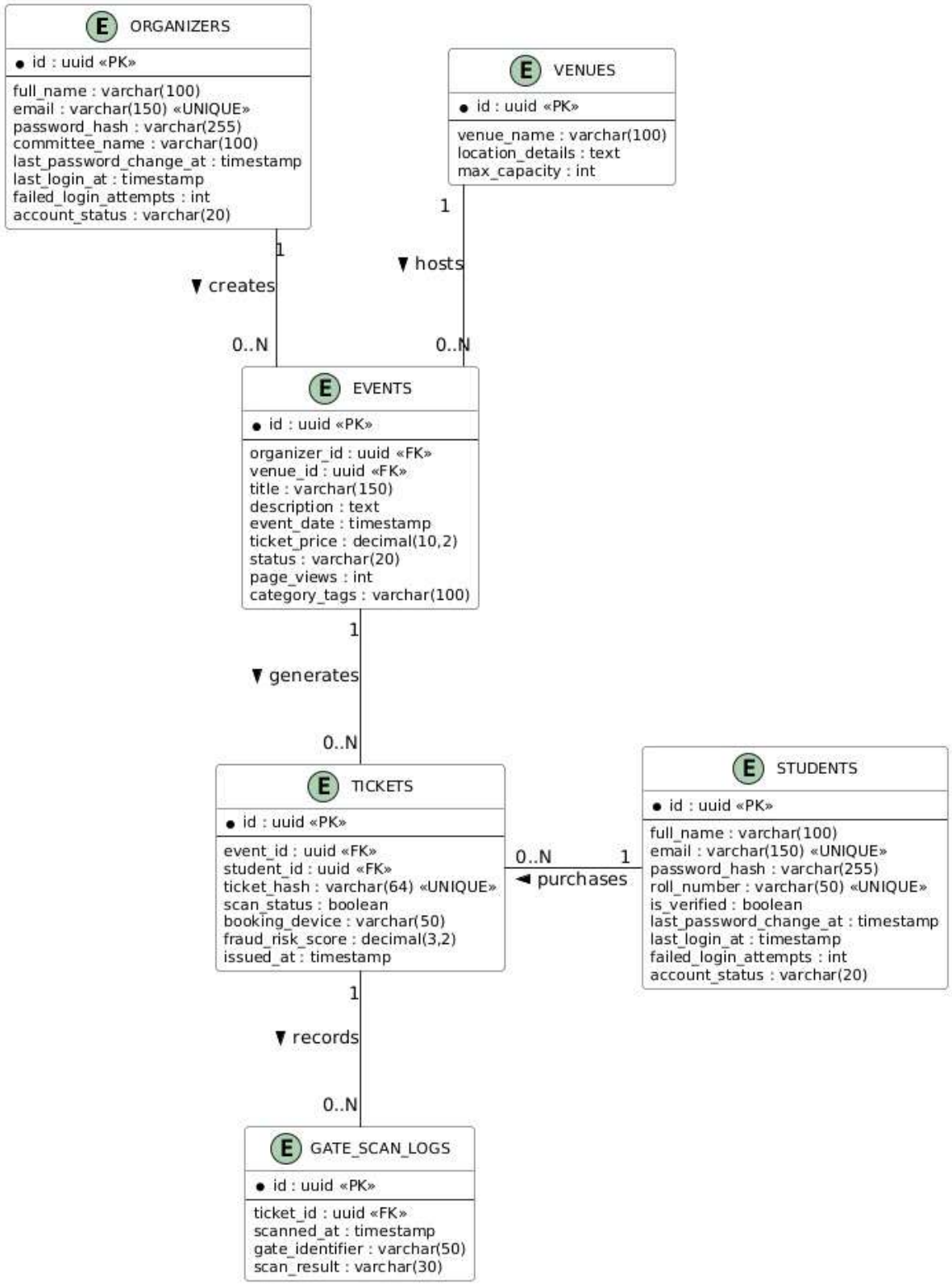


Fig. 4.4 ER Diagram

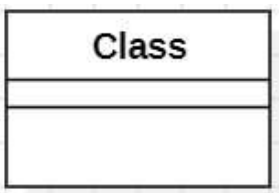
Relationships:

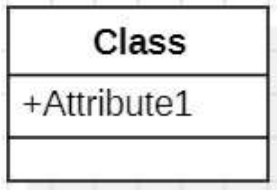
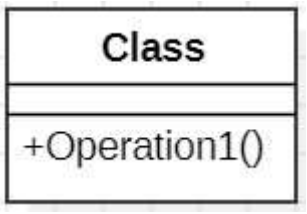
- **Organizer to Event: One-to-Many** (An organizer can create and manage multiple events, but each event belongs to exactly one organizer).
- **Venue to Event: One-to-Many** (A venue can host multiple events over time, but each event is assigned to one specific venue location).
- **Student to Ticket: One-to-Many** (A student can book multiple event tickets, but each individual ticket record is owned by exactly one student).
- **Event to Ticket: One-to-Many** (An event can have multiple tickets issued, but each ticket corresponds to one specific event).
- **Ticket to Gate_Scan_Log: One-to-Many** (A ticket can be logged across multiple scan attempts at entry gates, while each scan log entry references one specific ticket).

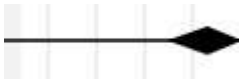
4.4 Class Diagram

A Class Diagram represents the static architecture of UniTik, depicting core objects, properties, and methods across modules such as authentication controllers, database connection handlers, QR code builders, and mailer utilities.

Table 4.4 Class Diagram Symbols

Symbol	Name	Description
	Class	Represents a class containing attributes and methods of an object.

	Attribute	Represents the data or properties associated with a class.
	Operation	Represents an operation or behaviour performed by a class.
	Association	Represents a relationship between two classes.

	Composition	Represents a strong whole-part relationship where the part depends on the whole.

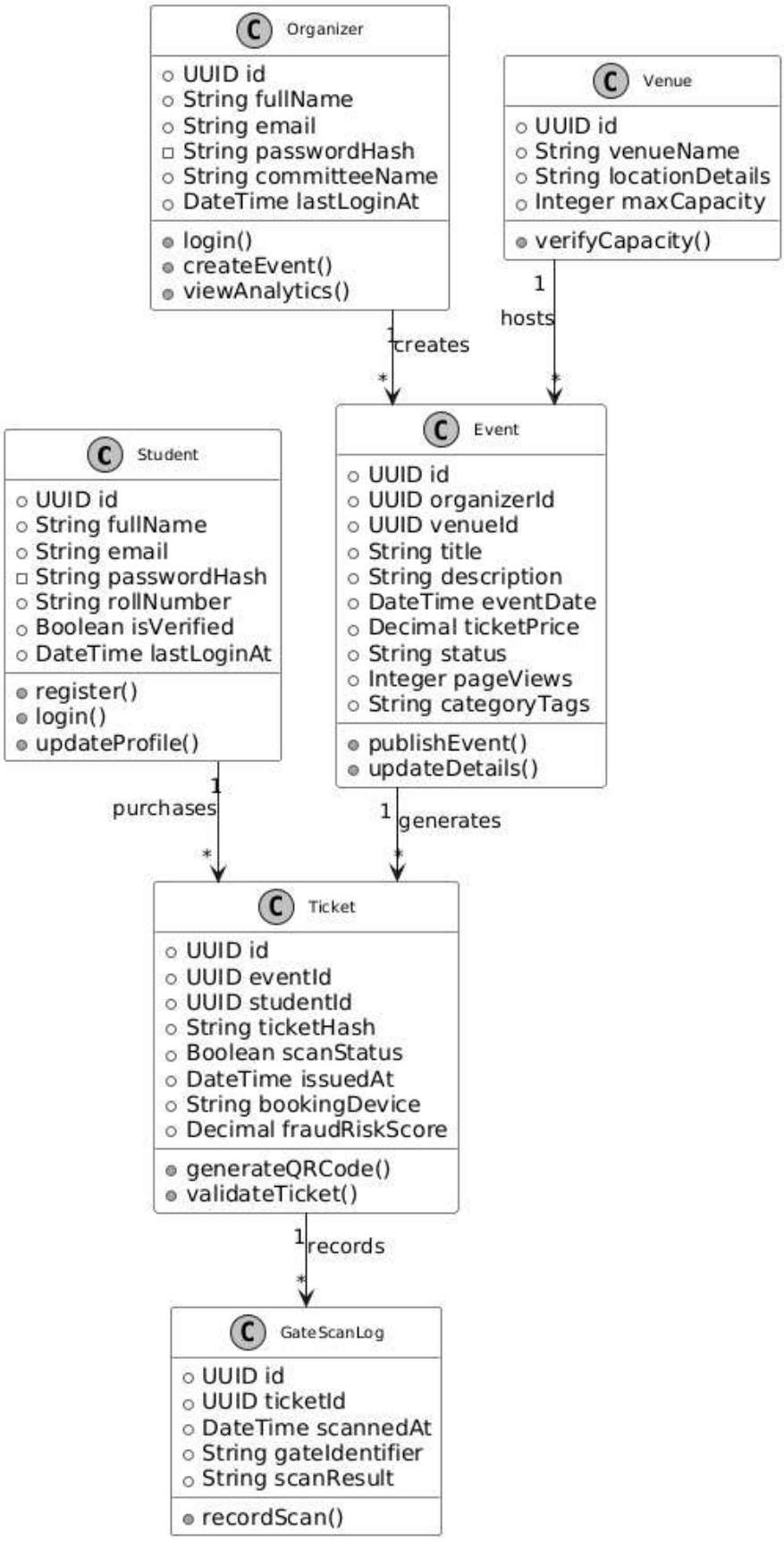
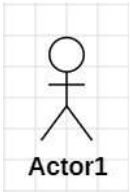
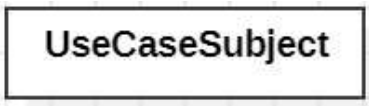
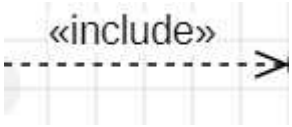
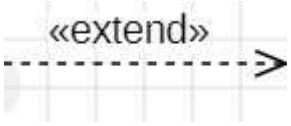


Fig. 4.5 Class Diagram

4.5 Use Case Diagram

A Use Case Diagram outlines the functional requirements of UniTik from the perspective of its distinct user roles: Students and Administrators (Organizers).

Table 4.5 Use Case Diagram Symbols

Symbol	Name	Description
	Actor	Represents a user or external entity that interacts with the system, such as Customer or Administrator.
	Use Case Subject	Represents the system being modeled and defines the scope of the use cases. In GroupCart, it represents the boundary of the GroupCart system.
	Use Case	Represents a function or service provided by the GroupCart system that can be performed by an actor.
	Association	Represents the interaction or communication between an actor and a use case.
	Include	Represents a mandatory relationship where one use case always includes the behavior of another use case.
	Extend	Represents an optional or conditional behavior that extends a base use case under specific conditions.

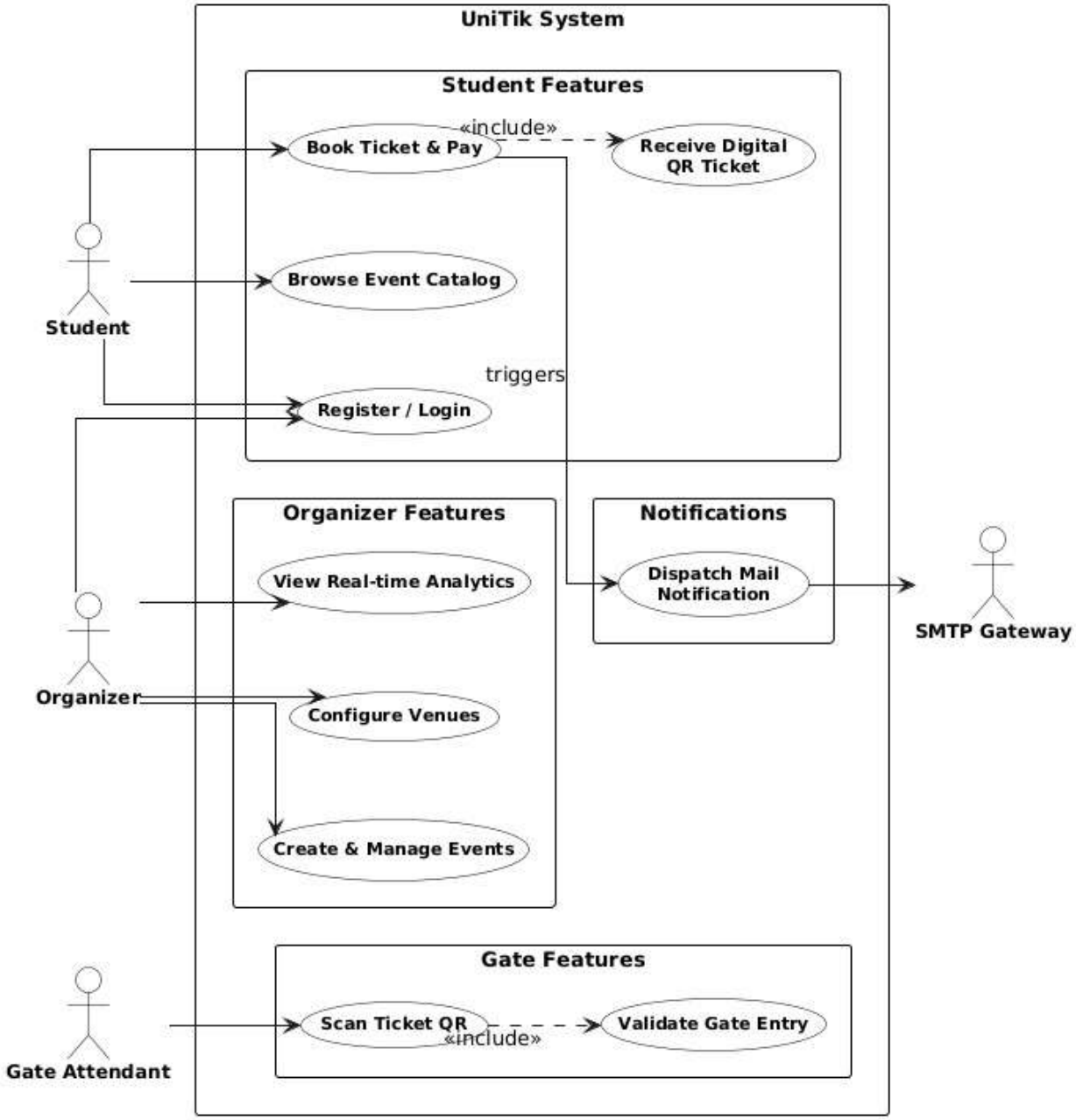


Fig. 4.7 Use Case Diagram

Actor – Use Case Relationships:

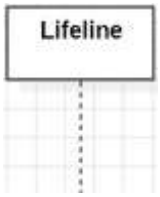
- STUDENT → (Register): The student creates a new account with their name, campus email, and roll number.
- STUDENT → (Login): The student authenticates securely to access the student portal.
- STUDENT → (Browse Events): The student explores available campus events and details.
- STUDENT → (Book Ticket): The student purchases or reserves a ticket for an event.

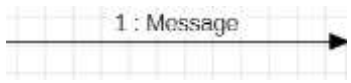
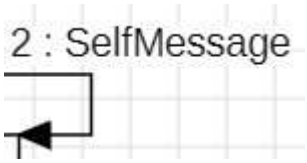
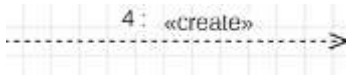
- Includes → (Generate QR Code): Booking a ticket automatically triggers unique hash generation and QR code creation.
- Includes → (Send Ticket Email): Booking automatically emails the digital pass and QR code to the student.
- STUDENT → (View Ticket Wallet): The student reviews their active purchased passes and QR codes.
- ADMIN → (Organizer Login): The administrator authenticates via a restricted portal using organizer credentials.
- ADMIN → (Manage Events): The administrator creates, updates, or configures campus events and venue allocations.
- ADMIN → (Gate Scanner): The administrator or gatekeeper scans student ticket QR codes to validate entry.
- Includes → (Log Scan Result): Scanning a ticket automatically records a timestamped entry in the gate scan logs.
- ADMIN → (View Analytics): The administrator monitors ticket sales, capacity utilization, and attendance metrics.

4.6 Sequence Diagram

A Sequence Diagram models the time-ordered exchange of messages between actors, Next.js frontend components, PHP backend endpoints, the Supabase PostgreSQL database, and external email libraries during critical workflows such as ticket generation.

Table 4.6 Sequence Diagram Symbols

Symbol	Name	Description
	Lifeline	Represents an object or participant involved in the interaction.

	Message	Represents a request or communication sent from one participant to another.
	Self Message	Represents an operation performed by an object on itself.
	Reply Message	Represents the response returned by the receiving participant.
	Create Message	Represents the creation of a new object or participant during the interaction.
	Activation	Represents the period during which a participant is performing an operation.

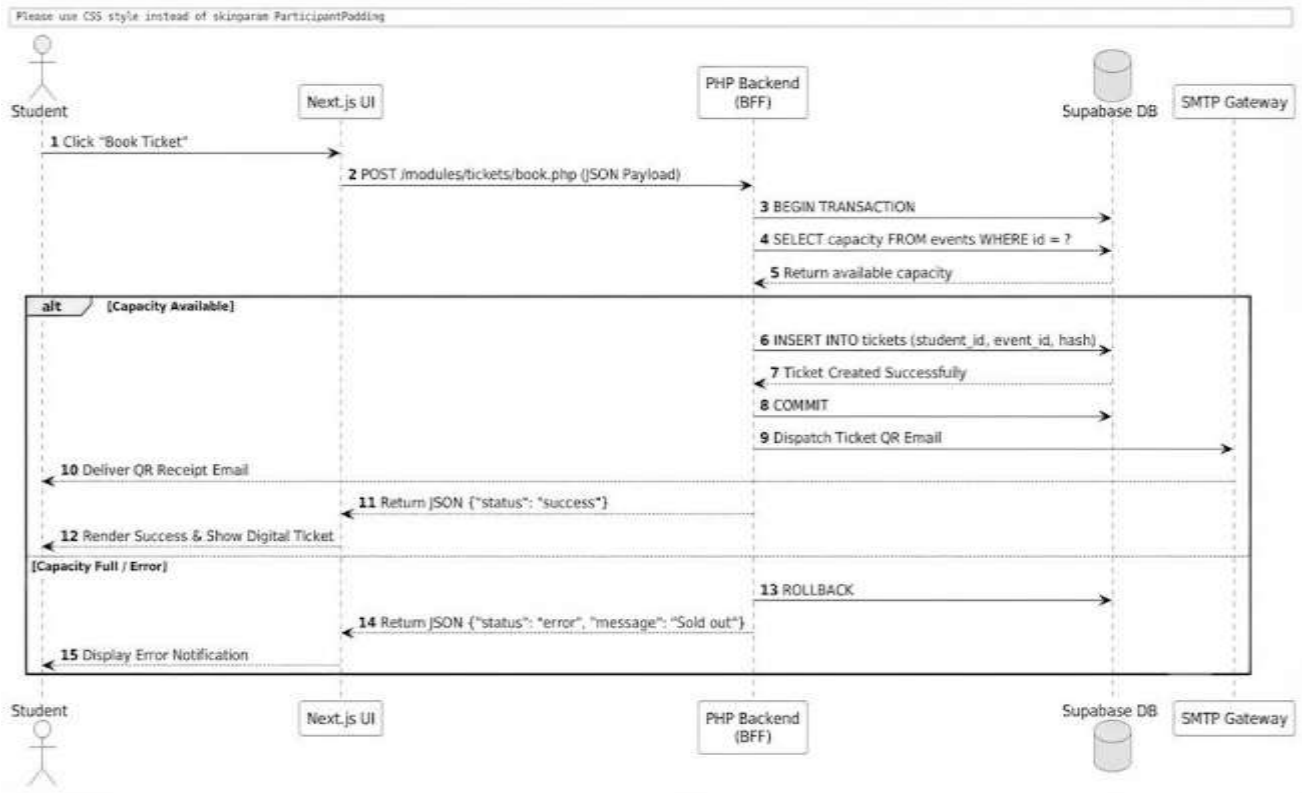


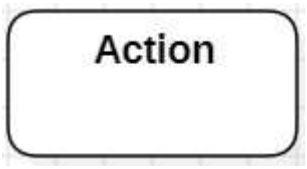
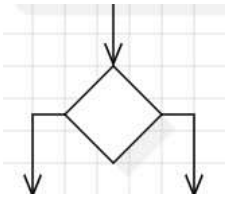
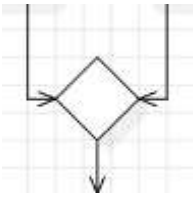
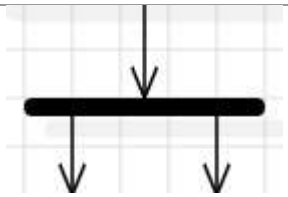
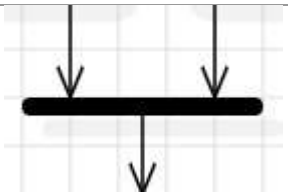
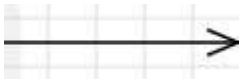
Fig. 4.8 Sequence Diagram

4.7 Activity Diagram

An Activity Diagram represents the procedural workflow and decision logic of system operations, such as the real-time ticket scanning sequence where the system validates ticket hashes, checks previous scan statuses, and grants or denies gate entry.

Table 4.7 Activity Diagram Symbols

Symbol	Name	Description
	Initial Node	Represents the starting point of the activity or workflow.

	Action	Represents an individual activity or operation performed by the user or system.
	Decision	Represents a condition that determines which path the workflow should follow.
	Merge	Combines alternative paths into a single flow.
	Fork	Splits one flow into multiple parallel activities.
	Join	Combines multiple parallel activities into a single flow.
	Control Flow	Shows the sequence and direction of movement between activities.
	Final Node	Represents the end of the activity or workflow.

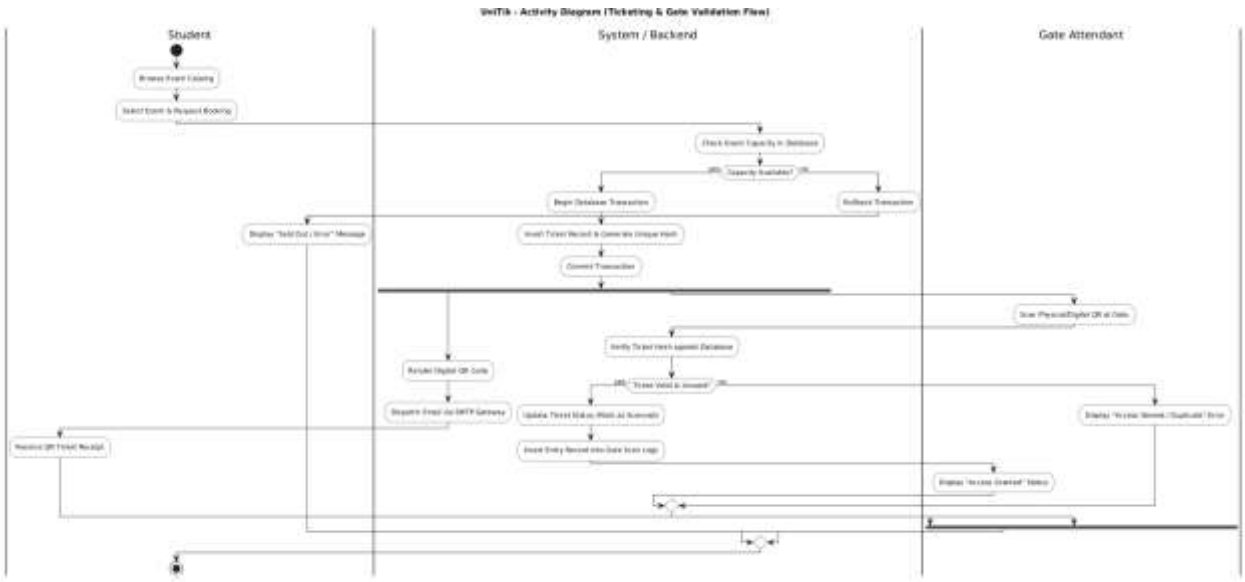


Fig. 4.9 Activity Diagra

REFERENCES

1. Mozilla Developer Network (MDN). HTML, CSS, JavaScript and Web APIs Documentation.
2. React. React Documentation.
3. Python Software Foundation. Python 3 Documentation.
4. FastAPI. FastAPI Documentation.
5. Oracle. MySQL Documentation.
6. Git. Git Documentation.
7. Object Management Group (OMG). Unified Modeling Language (UML) Specification, Version 2.5.1.
8. Wang, J. J., Zhao, X., & Li, J. J. Group Buying: A Strategic Form of Consumer Collective. Journal of Retailing, Vol. 89, No. 3, pp. 338–351, 2013.
9. Chow, C. W. C., Chow, C. S. F., Lai, J. Y. M., & Zhang, L. L. Online Group-Buying: The Effect of Deal Popularity on Consumer Purchase Intention. Journal of Consumer Behaviour, 2022.